

Practical 1: Joint Probability

Aim:

To compute and understand the joint probability distribution of two discrete random variables using Python, given a sample dataset.

Theory:

Joint probability is the probability that two (or more) events occur simultaneously. For two discrete random variables A and B, the joint probability is written as:

$$P(A = a, B = b) = (\text{number of times } A=a \text{ and } B=b \text{ occur together}) / (\text{total number of observations})$$

Key properties of a joint probability distribution:

- Every joint probability value lies between 0 and 1.
- The sum of all joint probabilities across every combination of outcomes equals 1.
- Joint distributions are the basis for deriving marginal probabilities (by summing over one variable) and conditional probabilities (used in Practical 2).

In this practical, we use a small weather dataset with two categorical variables — Weather (Sunny, Rainy, Overcast) and Play (Yes, No) — and compute their joint probability table using a frequency cross-tabulation (pandas.crosstab).

Code:

```
"""
```

Practical 1: Joint Probability

```
-----
```

We compute the joint probability distribution of two discrete random variables — Weather (Sunny/Rainy/Overcast) and Play (Yes/No) — from a small dataset, using pandas' crosstab functionality.

```
"""
```

```
import pandas as pd
```

```
# Sample dataset: 14 observations of Weather and whether a game was played
```

```
data = {
```

```
    "Weather": ["Sunny", "Sunny", "Overcast", "Rainy", "Rainy", "Rainy",
```

```
               "Overcast", "Sunny", "Sunny", "Rainy", "Sunny", "Overcast",
```

```
               "Overcast", "Rainy"],
```

```

    "Play": ["No", "No", "Yes", "Yes", "Yes", "No", "Yes", "No",
             "Yes", "Yes", "Yes", "Yes", "Yes", "No"]
}

```

```
df = pd.DataFrame(data)
```

```
n = len(df)
```

```
print("Dataset:")
```

```
print(df)
```

```
print(f"\nTotal number of observations (n) = {n}\n")
```

```
# ---- Joint Probability Table ----
```

```
#  $P(\text{Weather} = w, \text{Play} = p) = \text{count}(w, p) / n$ 
```

```
joint_counts = pd.crosstab(df["Weather"], df["Play"])
```

```
joint_prob = joint_counts / n
```

```
print("Joint Frequency (Count) Table:")
```

```
print(joint_counts)
```

```
print("\nJoint Probability Table P(Weather, Play):")
```

```
print(joint_prob.round(4))
```

```
# ---- A few specific joint probabilities ----
```

```
p_sunny_yes = joint_prob.loc["Sunny", "Yes"]
```

```
p_rainy_no = joint_prob.loc["Rainy", "No"]
```

```
print(f"\nP(Weather=Sunny, Play=Yes) = {p_sunny_yes:.4f}")
```

```
print(f"P(Weather=Rainy, Play=No) = {p_rainy_no:.4f}")
```

```
# Sanity check: all joint probabilities must sum to 1
```

```
print(f"\nSum of all joint probabilities = {joint_prob.values.sum():.4f}")
```

Output:

```
... Dataset:
      Weather Play
0      Sunny  No
1      Sunny  No
2  Overcast  Yes
3      Rainy  Yes
4      Rainy  Yes
5      Rainy  No
6  Overcast  Yes
7      Sunny  No
8      Sunny  Yes
9      Rainy  Yes
10     Sunny  Yes
11  Overcast  Yes
12  Overcast  Yes
13     Rainy  No

Total number of observations (n) = 14

Joint Frequency (Count) Table:
Play      No  Yes
Weather
Overcast   0   4
Rainy      2   3
Sunny      3   2

Joint Probability Table P(Weather, Play):
Play      No      Yes
Weather
Overcast  0.0000  0.2857
Rainy     0.1429  0.2143
Sunny     0.2143  0.1429

P(Weather=Sunny, Play=Yes) = 0.1429
P(Weather=Rainy, Play=No)  = 0.1429

Sum of all joint probabilities = 1.0000
```

Practical 2: Conditional Probability

Aim:

To compute conditional probabilities from a joint probability distribution using Python, and verify the relationship between joint, marginal, and conditional probabilities.

Theory:

Conditional probability measures the probability of an event A occurring, given that another event B has already occurred:

$$P(A | B) = P(A \text{ and } B) / P(B), \text{ provided } P(B) > 0$$

This is one of the foundational formulas of probabilistic reasoning and underlies many AI techniques, including Naive Bayes classifiers, Bayesian networks, and Hidden Markov Models. It allows a system to update its belief about one variable after observing evidence about another.

In this practical, we reuse the Weather/Play dataset from Practical 1. We first compute the marginal probabilities $P(\text{Weather})$ and $P(\text{Play})$ by summing over the joint table, and then divide the joint probabilities by the relevant marginal to obtain the conditional probability table $P(\text{Play} | \text{Weather})$.

Code:

```
"""
```

Practical 2: Conditional Probability

```
-----
```

Using the same Weather/Play dataset, we compute conditional probabilities such as $P(\text{Play}=\text{Yes} | \text{Weather}=\text{Sunny})$ using the formula:

$$P(A | B) = P(A \text{ and } B) / P(B)$$

```
"""
```

```
import pandas as pd
```

```
data = {
    "Weather": ["Sunny", "Sunny", "Overcast", "Rainy", "Rainy", "Rainy",
               "Overcast", "Sunny", "Sunny", "Rainy", "Sunny", "Overcast",
               "Overcast", "Rainy"],
```

```

    "Play": ["No", "No", "Yes", "Yes", "Yes", "No", "Yes", "No",
            "Yes", "Yes", "Yes", "Yes", "Yes", "No"]
}

df = pd.DataFrame(data)
n = len(df)

# Joint counts and joint probabilities (as in Practical 1)
joint_counts = pd.crosstab(df["Weather"], df["Play"])
joint_prob = joint_counts / n

# Marginal probability of each Weather category: P(Weather = w)
marginal_weather = df["Weather"].value_counts() / n
print("Marginal Probability P(Weather):")
print(marginal_weather.round(4))

# Marginal probability of each Play category: P(Play = p)
marginal_play = df["Play"].value_counts() / n
print("\nMarginal Probability P(Play):")
print(marginal_play.round(4))

# ---- Conditional Probability Table: P(Play | Weather) ----
# Divide each row of the joint table by that row's marginal P(Weather)
cond_play_given_weather = joint_prob.div(marginal_weather, axis=0)

print("\nConditional Probability Table P(Play | Weather):")
print(cond_play_given_weather.round(4))

# ---- Specific conditional probability calculations ----
p_yes_given_sunny = joint_prob.loc["Sunny", "Yes"] / marginal_weather["Sunny"]

```

```
p_no_given_rainy = joint_prob.loc["Rainy", "No"] / marginal_weather["Rainy"]
```

```
print(f"\nP(Play=Yes | Weather=Sunny) = "
```

```
    f'{joint_prob.loc['Sunny','Yes']:.4f} / {marginal_weather['Sunny']:.4f} "
```

```
    f"= {p_yes_given_sunny:.4f}")
```

```
print(f"P(Play=No | Weather=Rainy) = "
```

```
    f'{joint_prob.loc['Rainy','No']:.4f} / {marginal_weather['Rainy']:.4f} "
```

```
    f"= {p_no_given_rainy:.4f}")
```

```
# Sanity check: each row of the conditional table should sum to 1
```

```
print("\nRow sums of conditional table (should each be 1.0):")
```

```
print(cond_play_given_weather.sum(axis=1).round(4))
```

Output:

```
*** Marginal Probability P(Weather):
Weather
Sunny      0.3571
Rainy      0.3571
Overcast   0.2857
Name: count, dtype: float64

Marginal Probability P(Play):
Play
Yes      0.6429
No       0.3571
Name: count, dtype: float64

Conditional Probability Table P(Play | Weather):
Play      No  Yes
Weather
Overcast  0.0  1.0
Rainy     0.4  0.6
Sunny     0.6  0.4

P(Play=Yes | Weather=Sunny) = 0.1429 / 0.3571 = 0.4000
P(Play=No  | Weather=Rainy) = 0.1429 / 0.3571 = 0.4000

Row sums of conditional table (should each be 1.0):
Weather
Overcast    1.0
Rainy       1.0
Sunny       1.0
dtype: float64
```

Practical 3: Prolog Rule-Based System

Aim:

To implement a rule-based logical inference system (in the style of Prolog) using Python, demonstrating facts, rules, and queries resolved through unification.

Theory:

Prolog ("Programming in Logic") is a declarative logic-programming language built on three core ideas:

- Facts — unconditional statements about the world, e.g. `parent(john, mary)`.
- Rules — conditional statements that derive new facts from existing ones, using the `:-` ("if") operator, e.g. `grandparent(X, Y) :- parent(X, Z), parent(Z, Y)`.
- Queries — questions posed to the knowledge base, answered by the Prolog engine through backward-chaining resolution and unification of logic variables.

A standard Prolog interpreter (e.g., SWI-Prolog) is not part of a typical Python environment. Rather than approximate this with plain `if/else` code, this practical uses the `pyDatalog` library, which is a genuine Datalog/Prolog-style logic-programming engine embedded in Python — it supports real facts, real inference rules (defined with `<=`), and real unification-based queries, so the reasoning demonstrated is authentic logical inference, not a hand simulation.

Example domain — family relationships:

Facts: `parent(X, Y)`

Rules: `grandparent(X, Y) :- parent(X, Z), parent(Z, Y)`.

`sibling(X, Y) :- parent(Z, X), parent(Z, Y), X \= Y`.

Code:

```
"""
```

Practical 3: Prolog Rule-Based System

Prolog is a logic-programming language built around FACTS, RULES, and QUERIES, and it answers queries using backward-chaining resolution and unification. Pure Prolog interpreters are not part of the standard Python environment, so here we use the `pyDatalog` library, which brings genuine Prolog/Datalog-style logic programming (facts, rules with `<=`, and recursive/derived relations resolved via unification) directly into Python. This is not a hand-written imitation — it is an

actual logical inference engine operating on facts and rules, just invoked from Python syntax instead of a `.pl` file.

Family-relationship example (a classic Prolog demonstration):

Facts: `parent(X, Y)`

Rules: `grandparent(X, Y) :- parent(X, Z), parent(Z, Y).`

`sibling(X, Y) :- parent(Z, X), parent(Z, Y), X \== Y.`

"""

```
from pyDatalog import pyDatalog
```

```
# Declare the predicates (relations) and logic variables we will use
```

```
pyDatalog.create_terms('parent, grandparent, sibling, X, Y, Z')
```

```
# ---- FACTS: base parent-child relationships ----
```

```
+parent('john', 'mary')
```

```
+parent('john', 'mike')
```

```
+parent('mary', 'ann')
```

```
+parent('mary', 'tom')
```

```
+parent('mike', 'sam')
```

```
# ---- RULES: derived relationships defined in terms of parent/2 ----
```

```
# grandparent(X, Y) :- parent(X, Z), parent(Z, Y).
```

```
grandparent(X, Y) <= (parent(X, Z) & parent(Z, Y))
```

```
# sibling(X, Y) :- parent(Z, X), parent(Z, Y), X != Y.
```

```
sibling(X, Y) <= (parent(Z, X) & parent(Z, Y) & (X != Y))
```

```
# ---- QUERIES ----
```



```
print("All known parent(X, Y) facts:")  
print(parent(X, Y))
```

```
print("\nAll grandparent(X, Y) relationships inferred by the rule engine:")  
print(grandparent(X, Y))
```

```
print("\nWho are John's grandchildren? -> grandparent('john', Y)")  
print(grandparent('john', Y))
```

```
print("\nAll sibling(X, Y) pairs inferred by the rule engine:")  
print(sibling(X, Y))
```

```
print("\nIs 'mary' a grandparent of 'sam'? (query with both args bound)")  
result = grandparent('mary', 'sam')  
print("Yes, this holds." if result else "No, this does not hold.")
```

Output:

All known parent(X, Y) facts:

X		Y
----- -----		
mike		sam
mary		tom
mary		ann
john		mike
john		mary

All grandparent(X, Y) relationships inferred by the rule engine:

X		Y
----- -----		
john		sam
john		tom
john		ann

Who are John's grandchildren? -> grandparent('john', Y)

Y

sam
tom
ann

All sibling(X, Y) pairs inferred by the rule engine:

X		Y
----- -----		
tom		ann
ann		tom
mike		mary
mary		mike

Is 'mary' a grandparent of 'sam'? (query with both args bound)

No, this does not hold.

Practical 4: AIML System

Aim:

To build a simple conversational agent (chatbot) using AIML (Artificial Intelligence Markup Language) and demonstrate pattern-based response generation in Python.

Theory:

AIML is an XML-based markup language, originally developed for the A.L.I.C.E. chatbot, used to represent conversational knowledge in a structured way. Its knowledge base is built from `<category>` elements, each with:

- `<pattern>` — the input pattern to match against the user's message (may include wildcards such as `*` to match one or more words).
- `<template>` — the response the bot returns when its pattern is matched.

At runtime, an AIML interpreter (Kernel) loads one or more `.aiml` files into a pattern-matching data structure and, for each user utterance, finds the most specific matching pattern and returns the associated template. This is a form of rule-based, retrieval-style natural language processing — the bot does not "understand" language in a deep sense; it performs structured pattern matching against pre-authored rules.

This practical uses the `python-aiml` library, a genuine AIML interpreter for Python (it parses `.aiml` XML, builds the matching graph, and resolves wildcards), so the chatbot behaviour shown is real AIML execution rather than a re-implementation from scratch.

Code:

```
"""
```

Practical 4: AIML (Artificial Intelligence Markup Language) System

AIML is an XML dialect used to define pattern-matching conversational agents (chatbots). An AIML knowledge base consists of `<category>` elements, each containing:

- `<pattern>` - the (possibly wildcard-containing) input pattern
- `<template>` - the response to return when the pattern matches

We use the ``python-aiml`` library, which implements a genuine AIML interpreter (pattern matching, wildcards, graph-based knowledge storage) so the chatbot logic below is real AIML execution, not a

hand-rolled simulation.

Steps:

1. Define a small AIML knowledge base (categories) in a .aiml file.
2. Load it into an AIML Kernel.
3. Send sample user inputs and print the bot's matched responses.

```
"""
```

```
import aiml
```

```
import os
```

```
# ---- Step 1: Define the AIML knowledge base ----
```

```
aiml_content = """<?xml version="1.0" encoding="UTF-8"?>
```

```
<aiml version="1.0.1">
```

```
<category>
```

```
<pattern>HELLO</pattern>
```

```
<template>Hello! I am an AIML-based chatbot. How can I help you today?</template>
```

```
</category>
```

```
<category>
```

```
<pattern>WHAT IS YOUR NAME</pattern>
```

```
<template>My name is PyBot, a simple AIML demonstration agent.</template>
```

```
</category>
```

```
<category>
```

```
<pattern>WHAT IS AIML</pattern>
```

```
<template>AIML stands for Artificial Intelligence Markup Language. It is an XML-based language used
to create pattern-matching conversational agents.</template>
```

```
</category>
```

```
<category>
```

```
<pattern>* YOUR NAME *</pattern>
```

```
<template>My name is PyBot!</template>
```

```
</category>
```

```
<category>
```

```
<pattern>BYE</pattern>
```

```
<template>Goodbye! Have a nice day.</template>
```

```
</category>
```

```
</aiml>
```

```
"""
```

```
os.makedirs("aiml_data", exist_ok=True)
```

```
aiml_path = os.path.join("aiml_data", "basic.aiml")
```

```
with open(aiml_path, "w") as f:
```

```
    f.write(aiml_content)
```

```
# ---- Step 2: Create the AIML kernel and load the knowledge base ----
```

```
kernel = aiml.Kernel()
```

```
kernel.learn(aiml_path)
```

```
# ---- Step 3: Converse with the bot using sample inputs ----
```

```
sample_inputs = [
```

```
    "HELLO",
```

```
    "WHAT IS YOUR NAME",
```

```
    "WHAT IS AIML",
```

```

"DO YOU KNOW YOUR NAME PLEASE", # matched by the wildcard pattern
"BYE",
]

```

```

print("AIML Chatbot Demonstration\n" + "-" * 40)
for user_input in sample_inputs:
    response = kernel.respond(user_input)
    print(f"User : {user_input}")
    print(f"Bot : {response}\n")

```

Output:

```

Loading aiml_data/basic.aiml...done (0.03 seconds)
AIML Chatbot Demonstration
-----
User : HELLO
Bot  : Hello! I am an AIML-based chatbot. How can I help you today?

User : WHAT IS YOUR NAME
Bot  : My name is PyBot, a simple AIML demonstration agent.

User : WHAT IS AIML
Bot  : AIML stands for Artificial Intelligence Markup Language. It is an
XML-based language used to create pattern-matching conversational agents.

User : DO YOU KNOW YOUR NAME PLEASE
Bot  : My name is PyBot!

User : BYE
Bot  : Goodbye! Have a nice day.

```